

Chapter 4: System Design

4.1 Introduction

The System Design chapter represents the pivotal transition from abstract project requirements into a concrete, implementable technical architecture. While the requirements analysis phase establishes *what* the Smart Bus Garage Management System must accomplish — managing fleets, tracking drivers, automating gate access, monitoring vehicle health, and dispatching buses in response to real-time crowding conditions — the system design phase establishes *how* each of those goals is realized through specific hardware configurations, software components, database schemas, communication protocols, and interaction patterns.

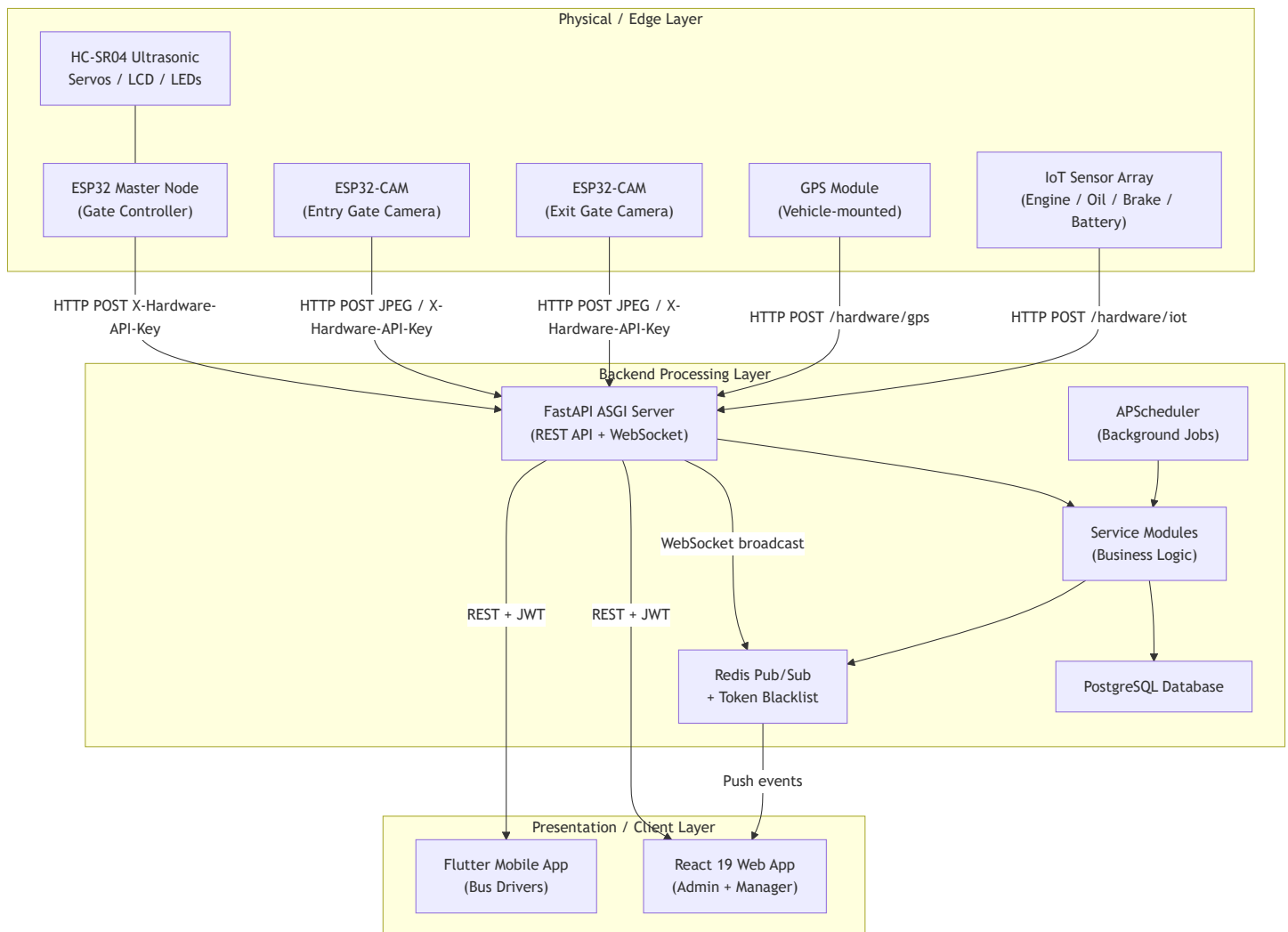
This chapter presents the complete technical blueprint of the Smart Bus Garage Management System: an integrated **IoT**-driven automation platform that connects physical garage infrastructure to a centralized cloud-capable backend and a dual-platform client layer — a **React**-based web application for administrative and managerial operations, and a **Flutter** mobile application for field-deployed bus drivers. The system is designed to address the operational complexities of a modern bus fleet, including driver rotation scheduling, automated shift management, sensor-based preventive maintenance, **Automatic Number Plate Recognition (ANPR)** for gate control, and real-time passenger crowding detection with autonomous extra-dispatch logic.

The architectural decisions documented here are informed by four core engineering principles: **modularity**, enabling independent development and testing of each subsystem; **scalability**, ensuring the system can accommodate growing fleets and user bases without architectural rework; **real-time responsiveness**, satisfying the latency requirements of IoT sensor ingestion and live operational alerts; and **security**, protecting both hardware communication channels and user-facing application surfaces through layered authentication, input validation, and audit mechanisms.

Each section of this chapter addresses a different layer of the system, from the physical hardware layer and microcontroller-driven sensing devices, through the asynchronous processing backend, to the end-user interface. Actual implementation artifacts — including **API** route paths, **SQLAlchemy** model definitions [2], database column configurations, business-logic service names, scheduled job timings, and middleware specifications — are cited throughout to provide an accurate and reproducible technical record of the system as built.

4.2 System Block Diagram

The Smart Bus Garage Management System is composed of three primary architectural layers that communicate along well-defined data pathways. These layers are the **Physical/Edge Layer**, the **Backend Processing Layer**, and the **Presentation/Client Layer**. Together they form a cohesive, end-to-end IoT pipeline.



4.2.1 Physical / Edge Layer

At the lowest level of the architecture sit the physical hardware components deployed within and around the bus garage facility. These components include **GPS** tracking modules installed on each vehicle, multi-parameter IoT sensor assemblies connected to vehicle diagnostics, ANPR camera units positioned at entry and exit gates, and overhead passenger-counting vision systems mounted inside bus cabins. Each hardware node collects raw operational data — coordinates, sensor readings,

license plate strings, and headcount figures — and transmits this data as structured **HTTP** POST requests [7] over the facility's Wi-Fi or cellular network to the backend processing layer.

Hardware devices authenticate to the backend using a shared cryptographic key transmitted in the `X-Hardware-API-Key` HTTP header [7]. This design isolates hardware-originated traffic from user-originated traffic and allows the backend to enforce separate rate limits and validation logic for each source.

4.2.2 Backend Processing Layer

The central processing layer is built on **FastAPI** [1], a Python **ASGI** web framework. It exposes a versioned **REST** API under the base path `/api/v1` and a persistent **WebSocket** endpoint at `/ws` [8]. The backend receives data from hardware nodes through four dedicated ingestion endpoints (`/hardware/gps`, `/hardware/iot`, `/hardware/anpr`, `/hardware/camera`), processes it through service modules containing domain-specific business logic, persists state to a **PostgreSQL** [10] relational database via **SQLAlchemy's** async **ORM** [2], and propagates real-time alerts to connected clients via a **Redis** [11] Pub/Sub-backed WebSocket bus [8].

An embedded **APScheduler** instance manages time-driven background tasks: daily rotation schedule generation fires at 05:30 each morning, a rotation-processing job runs every five minutes to manage driver swapping logic, and a midnight reset job synchronizes daily operational counters.

4.2.3 Presentation / Client Layer

The topmost layer is a **dual-platform client** that separates operational roles across two distinct applications. The **React 19** [16] single-page web application, bundled with **Vite** [15], serves **Administrator** and **Fleet Manager** users from a browser. It communicates with the backend via an **Axios** HTTP client [7] configured with **JWT** Bearer token injection [9] and maintains a persistent **WebSocket** [8] connection for real-time operational alerts. Separately, the **Flutter** mobile application — implemented in **Dart** — serves **Bus Driver** users on Android mobile devices. It communicates with the backend exclusively via **Dio** (an HTTP client library), using **BLoC** (Business Logic Component) for state management and **SharedPreferences** for local token persistence. The Flutter app does not implement a WebSocket connection in the current release; real-time push notifications are planned for a future iteration.

4.2.4 Inter-Layer Data Flow

Data flows through the system along two primary pathways:

- **IoT Ingest Pathway:** Hardware device → HTTP POST [7] (API Key auth) → FastAPI [1] hardware route → Service validation → SQLAlchemy [2] write → Conditional WebSocket broadcast [8] via Redis [11] → Manager/Admin browser client.
- **User Interaction Pathway:** React [16] client → HTTP request [7] (JWT Bearer auth [9]) → FastAPI [1] application route → SQLAlchemy [2] read/write → JSON response → React state update → UI re-render.

These two pathways converge at the database and at the WebSocket broker [8], which serves as the real-time notification fabric that keeps the operational dashboard synchronized with physical events without requiring polling.

4.3 System Architecture Design

4.3.1 Architectural Pattern

The Smart Bus Garage Management System follows a **layered IoT Client-Server architecture** with elements of the **service-oriented** pattern at the backend layer. This choice is deliberate and directly motivated by the system's requirements.

A pure client-server model would suffice for administrative CRUD operations, but the system must also ingest continuous streams of sensor data from autonomous hardware nodes, respond to threshold violations within seconds, and simultaneously notify multiple human operators in real time. A monolithic architecture would conflate these concerns, making it difficult to independently scale the hardware-ingestion path from the user-facing API. Conversely, a full microservices decomposition would introduce operational overhead — service discovery, inter-service messaging, distributed tracing — disproportionate to a graduation-scale system.

The chosen architecture strikes the appropriate balance: a **single deployable FastAPI [1] application** that internally separates concerns into distinct route modules, service classes, and background workers, all sharing a single PostgreSQL [10] database and Redis [11] broker. This preserves operational simplicity while maintaining the logical modularity that would facilitate a future microservices decomposition if deployment scale demands it.

4.3.2 Why FastAPI as the Backend Framework

FastAPI [1] is selected as the backend framework for several technically justified reasons:

- **Asynchronous I/O by design:** FastAPI [1] is built on **Starlette** and employs Python's `asyncio` event loop natively. All database queries are executed through `asyncpg` (PostgreSQL's [10] async driver) with SQLAlchemy's [2] async session model, meaning that a single worker process can handle thousands of concurrent IoT sensor submissions and API requests without thread-blocking.
- **Automatic OpenAPI documentation:** FastAPI [1] generates interactive Swagger UI documentation from Pydantic [14] schema definitions at no additional development cost, which accelerates integration testing of the hardware ingestion endpoints.
- **Pydantic v2 validation** [14]: All request payloads are validated at the framework boundary through Pydantic models before any service logic is invoked, providing a strong defense against malformed hardware data.
- **WebSocket support** [8]: FastAPI's [1] `websockets` integration, built on Starlette, allows persistent bidirectional connections to be managed in the same async event loop as REST handlers, eliminating the need for a separate WebSocket server process.

4.3.3 Why PostgreSQL as the Database

PostgreSQL [10] is chosen over alternative data stores for the following reasons:

- The system's data model is highly relational: drivers are linked to vehicles, vehicles to garages, trips to rotation assignments, and maintenance requests to vehicles and approving managers. PostgreSQL's [10] foreign key constraints and join query capabilities are architecturally appropriate for this domain.
- The `asyncpg` driver provides native `async/await` semantics, allowing the database layer to participate in FastAPI's [1] async I/O model without performance degradation.
- PostgreSQL's [10] `JSONB` column type is used for audit log `old_values` and `new_values` fields, providing schemaless storage for change-history payloads without abandoning the relational schema for the rest of the model.
- Alembic, the SQLAlchemy [2] migration framework, provides version-controlled schema evolution, which is essential for iterative development on a multi-model project.

4.3.4 Why Redis as the Pub/Sub Broker

The WebSocket [8] notification system requires that alerts originating from hardware ingestion — crowding events, IoT threshold breaches, gate authorization results — be forwarded to all currently connected web clients regardless of which server process handled the original HTTP request [7]. Redis [11] Pub/Sub provides a lightweight, battle-tested message broker that decouples the alert publisher (hardware route handlers) from the alert consumers (WebSocket connection manager [8]). Redis [11] also serves as the backend for the JWT [9] token blacklist: when a user changes their

password, the invalidated token's **JTI** is stored in Redis [11] with a **TTL** matching the token's remaining validity window, preventing reuse without requiring a database query on every protected request.

4.4 System Components

The Smart Bus Garage Management System is decomposed into the following core logical modules, each with distinct responsibilities, inputs, and outputs.

4.4.1 Authentication and Authorization Module

Responsibility: This module governs all aspects of identity verification and access control across the system. It is implemented across `/app/api/auth.py` (route handlers), `/app/core/security.py` (cryptographic primitives), and `/app/api/deps.py` (FastAPI [1] dependency injection).

The module provides four public endpoints: `POST /api/v1/auth/login` for credential-based token issuance, `POST /api/v1/auth/refresh` for access token renewal using a long-lived refresh token, `GET /api/v1/auth/me` for retrieving the authenticated user's profile, and `PATCH /api/v1/auth/me` for profile updates. Passwords are hashed at rest using **bcrypt** with a configurable work factor. Access tokens are **HS256**-signed JWTs [9] with a 60-minute expiry; refresh tokens carry a 7-day TTL. **Role-Based Access Control (RBAC)** is enforced via FastAPI [1] dependency functions (`get_current_admin`, `get_current_manager`, `get_current_driver`) injected into protected route handlers.

Inputs: User credentials (email, password), JWT Bearer tokens [9].

Outputs: Signed JWT access/refresh token pairs [9], user profile payloads, HTTP 401/403 rejection responses [7].

4.4.2 Hardware Data Ingestion Module

Responsibility: This module defines the four API endpoints through which physical hardware nodes submit operational data to the backend. It is implemented in `/app/api/hardware.py` and protected by API key authentication (`X-Hardware-API-Key` header validation [7]).

- `POST /api/v1/hardware/gps` — Receives GPS telemetry: `vehicle_id`, `latitude`, `longitude`, `speed`, `heading`. Writes a `GPSTracking` record and updates the vehicle's `current_latitude` / `current_longitude` fields in real time.
- `POST /api/v1/hardware/iot` — Receives multi-sensor vehicle diagnostics: `engine_temp_c`, `oil_pressure_psi`, `brake_pad_mm`, `battery_voltage`. Writes an `IotSensorReading` record and

evaluates each value against configured safety thresholds. If any threshold is exceeded, an `EMERGENCY` maintenance request is auto-created and a WebSocket [8] alert is broadcast to Manager and Admin roles via Redis [11].

- `POST /api/v1/hardware/anpr` — Receives ANPR results: `gate_id`, `plate_number`, `confidence`. Performs plate lookup and authorization logic, writes a `gateLog` record, and broadcasts a `gate_auth` WebSocket [8] event.
- `POST /api/v1/hardware/camera` — Receives YOLOv8 [13] passenger-counting results: `trip_id`, `passenger_count`, `confidence`. Computes a `crowding_score` against vehicle capacity and, if the score exceeds 90%, triggers the automated extra-dispatch workflow.

Inputs: Structured JSON payloads from hardware nodes, API key header [7].

Outputs: Database writes via SQLAlchemy [2], conditional WebSocket broadcasts [8], HTTP 200/401/422 responses [7].

4.4.3 Driver Operations Module

Responsibility: This module provides the complete operational interface for Bus Driver role users. It is implemented in `/app/api/drivers.py` and covers the full lifecycle of a driver's working day.

Drivers interact with the system through eleven endpoints: retrieving their assigned profile and active trips, starting and ending trips with validation logic (enforcing a 15-minute pre-start window and a minimum trip duration of 50% of the route's estimated time), managing break periods (with minimum 10-minute and maximum 30-minute durations enforced), performing shift check-in and check-out, submitting reroute requests, and retrieving their notification feed. All requests are validated at the boundary by Pydantic [14] schemas before reaching service logic.

Inputs: Authenticated JWT [9] (Driver role), trip/break action payloads.

Outputs: Trip state transitions, `BreakLog` records, `RerouteLog` records, notification payloads.

4.4.4 Fleet Manager Module

Responsibility: This module serves the Fleet Manager role, providing real-time operational oversight without the data-modification privileges of the Administrator. It is implemented in `/app/api/manager.py`.

The manager dashboard (`GET /api/v1/manager/dashboard/stats`) aggregates live fleet metrics. The fleet endpoint (`GET /api/v1/manager/fleet`) returns vehicle statuses with their latest GPS coordinates. Maintenance endpoints allow managers to approve or reject pending requests (`PATCH /api/v1/manager/maintenance/{id}/approve|reject`). Reroute approval endpoints (`PATCH /api/v1/manager/reroutes/{id}/approve|reject`) allow managers to authorize driver-requested

route changes. A dedicated notifications endpoint surfaces WebSocket [8]-delivered alerts in paginated form.

Inputs: Authenticated JWT [9] (Manager role), approval/rejection payloads.

Outputs: Aggregated statistics, fleet state snapshots, `MaintenanceRequest` status updates, `RerouteLog` status updates.

4.4.5 Administration Module

Responsibility: The Administrator module provides full system configuration and data management capabilities. It is implemented across `/app/api/admin.py` and sub-routers for users, drivers, vehicles, routes, tickets, rotations, and trips.

Administrators perform all CRUD operations on the core entity set: `User`, `Driver`, `Vehicle`, `Route` (including `RouteStop` management), and `Ticket`. They can trigger rotation schedule generation on demand (`POST /api/v1/admin/rotations/generate`), view all trip records with full detail, and access the complete audit log (`GET /api/v1/admin/audit-logs`) for compliance and incident investigation. All operations are protected by JWT [9] Bearer token validation and the `get_current_admin` FastAPI [1] dependency.

Inputs: Authenticated JWT [9] (Admin role), entity create/update payloads validated by Pydantic [14].

Outputs: Created/updated/deleted entity records, scheduled `RotationAssignment` and `Trip` records.

4.4.6 Rotation and Scheduling Service

Responsibility: The `rotation_service.py` module implements the core business logic of the driver rotation system. This service is invoked both by the `APScheduler` background job (daily at 05:30) and by the admin manual trigger endpoint.

For each active `Route` and each shift type (`MORNING`, `EVENING`), the service assigns three drivers (`DRIVER_1`, `DRIVER_2`, `DRIVER_3`) and two vehicles to a `RotationAssignment`. It generates `SCHEDULED` trip records for each driver covering their expected `OUTBOUND` and `INBOUND` runs within the shift window, calculating start times based on route `estimated_time_minutes`, a 15-minute inter-trip rest, and the configured `D3_SHIFT_OFFSET_HOURS` stagger. A separate rotation-processing job, firing every five minutes, evaluates the current time against each assignment's schedule and initiates driver exchanges at the appropriate handover points, logging each swap as a `DriverExchange` record. All database writes are performed through SQLAlchemy's [2] async session within a single transactional unit of work.

Inputs: Active routes, available `OFF_DUTY` drivers, `FREE` vehicles, current timestamp.

Outputs: `RotationAssignment` records, `Trip` records, `DriverExchange` records, driver status updates.

4.4.7 Maintenance Management Module

Responsibility: Implemented in `maintenance_service.py` and exposed through both admin and manager route handlers, this module manages the full lifecycle of vehicle maintenance requests from creation through completion.

Maintenance requests are created in two ways: manually by users submitting a request through the admin interface, and automatically by the IoT sensor threshold logic when a safety-critical sensor value is detected. The module enforces a structured approval workflow: requests begin in `PENDING` status, are moved to `APPROVED` or `REJECTED` by a manager, transition to `IN_PROGRESS` when work begins, and finally reach `COMPLETED` when resolved. The vehicle's `status` field is updated in parallel — e.g., set to `MAINTENANCE` upon approval — to prevent the rotation service from assigning the vehicle to new trips. All state changes are recorded by the audit service in the PostgreSQL [10] `audit_logs` table.

Inputs: `MaintenanceRequest` create/update payloads, IoT threshold breach events.

Outputs: `MaintenanceRequest` record state transitions, vehicle `status` updates, manager notifications.

4.4.8 Real-Time Notification and WebSocket Module

Responsibility: The `sockets.py` module implements the `ConnectionManager` class responsible for all persistent WebSocket [8] connections. It maintains three sets of active connections partitioned by user role (`ADMIN` , `MANAGER` , `DRIVER`), validates JWT [9] tokens on connection establishment, and enforces a per-connection rate limit of 10 messages per second.

Redis [11] Pub/Sub is used as the broadcast backbone: when a backend event handler (hardware route, rotation service, crowding service) needs to notify clients, it publishes to a Redis [11] channel. A background supervisor coroutine subscribes to this channel and relays messages to all WebSocket [8] connections matching the target role. This design ensures that WebSocket [8] delivery is decoupled from the request that triggered the event, and that the system remains correct when deployed across multiple ASGI worker processes.

Inputs: JWT [9]-authenticated WebSocket upgrade requests [8], Redis [11] Pub/Sub messages from internal services.

Outputs: JSON-framed WebSocket [8] messages to role-filtered client sets.

4.5 Hardware Integration

4.5.1 Overview of the Physical Layer

The physical layer of the Smart Bus Garage Management System constitutes the edge of the IoT pipeline, where real-world garage and vehicle conditions are sensed, quantified, and transmitted to the central backend. The hardware architecture is designed around two deployment contexts: **vehicle-mounted units** that travel with buses throughout their routes, and **fixed infrastructure units** installed at garage gates and within bus cabins.

All hardware nodes communicate with the backend over HTTP [7] using the RESTful hardware ingestion API described in Section 4.4.2. The use of HTTP [7] rather than MQTT [12] or a proprietary binary protocol is a deliberate design choice: it eliminates the need for an additional message broker (beyond Redis [11], which is already required for WebSocket [8] distribution), allows hardware authentication to reuse the same API key mechanism as other protected endpoints, and ensures that any IP-connected device capable of issuing HTTP POST requests [7] can serve as a hardware node without firmware-level changes to the protocol stack.

4.5.2 Vehicle Tracking Subsystem (GPS Module)

Each bus in the fleet is equipped with a GPS tracking module — compatible with UBLOX or SIM7600-series modules commonly interfaced to ESP32-class [5] microcontrollers — that periodically samples the vehicle's geographic coordinates, instantaneous speed in km/h, and compass heading. This data is packaged as a JSON payload and submitted to `POST /api/v1/hardware/gps` via HTTP [7]. The backend's `ingest_gps()` handler writes a `GPSTracking` record to the PostgreSQL [10] database (with columns `latitude`, `longitude`, `speed`, `heading`, and `recorded_at`) and simultaneously updates the parent `vehicle` record's `current_latitude` and `current_longitude` fields through SQLAlchemy's [2] async ORM, ensuring that any subsequent query for vehicle position reflects the most recent known location without requiring an aggregation query over the `gps_tracking` table.

The `gps_tracking` table is indexed on `(vehicle_id, recorded_at)` to support efficient time-series queries by vehicle, which are expected to be the predominant access pattern for route replay and driver performance reporting.

4.5.3 Vehicle Diagnostics Subsystem (IoT Sensor Array)

The IoT sensor subsystem monitors four critical vehicle health parameters in real time. Sensor readings are transmitted via HTTP [7] to `POST /api/v1/hardware/iot` and persist as `IotSensorReading`

records through SQLAlchemy [2]. The four monitored parameters and their associated safety thresholds are:

Sensor Parameter	Column	Emergency Threshold	Physical Measurement
Engine Temperature	engine_temp_c	> 105 °C	Thermocouple / NTC thermistor on engine block
Oil Pressure	oil_pressure_psi	< 25 PSI	Piezo-resistive pressure transducer on oil gallery
Brake Pad Wear	brake_pad_mm	< 3 mm	Inductive wear sensor on brake caliper
Battery Voltage	battery_voltage	< 11.8 V	Resistive voltage divider on 12 V bus

The ESP32 [5] microcontroller reads these sensors via its **GPIO**, **I2C**, **SPI**, or **UART** peripheral interfaces, depending on the sensor module type, as documented in the ESP32 Technical Reference Manual [5]. When any submitted value crosses its threshold, the backend's `ingest_iot()` handler automatically creates a `MaintenanceRequest` record with `type=EMERGENCY`, `status=PENDING`, and `priority=1` (the highest priority in the system's numeric priority scale). A WebSocket [8] alert of type `iot_alert` is simultaneously broadcast via Redis [11] to all connected `MANAGER` and `ADMIN` role clients, ensuring that operations staff are notified of safety-critical conditions within the WebSocket delivery latency window, without requiring them to poll a dashboard.

4.5.4 Automatic Number Plate Recognition (ANPR) Subsystem

ANPR cameras are installed at the garage entry and exit gates and are modeled in the PostgreSQL [10] database by the `GateCamera` entity (`__tablename__ = "gate_cameras"`), which stores each camera's `location_name`, `ip_address`, `gate_type` (`ENTRY`, `EXIT`, or `BOTH`), and operational status. When a vehicle approaches a gate, the ANPR system captures the license plate, performs on-device optical character recognition, and submits the result to `POST /api/v1/hardware/anpr` via HTTP [7] with the fields `gate_id`, `plate_number`, and `confidence`.

The backend's `ingest_anpr()` handler applies the following authorization logic:

1. If `confidence < 0.85`, the event is classified as `IGNORED` — the recognition quality is insufficient for a binding access decision, and a `GateLog` record is written with event type `IGNORED`.

2. If `confidence ≥ 0.85` , the handler queries the `vehicles` table via SQLAlchemy [2] for a matching `plate_number` .
3. If a matching vehicle is found and its `status` is one of `{EN_ROUTE, ASSIGNED, FREE}` , the event is classified as `GRANTED` .
4. In all other cases (unrecognized plate, vehicle in `MAINTENANCE` or `OUT_OF_SERVICE` status), the event is classified as `DENIED` .

All events, regardless of outcome, are persisted as `GateLog` records and broadcast as `gate_auth` WebSocket [8] events to Manager and Admin clients, providing a complete audit trail of gate activity.

4.5.5 Passenger Counting Subsystem (Computer Vision Camera)

The passenger occupancy monitoring subsystem employs cameras mounted inside bus cabins running a **YOLOv8** [13]-based object detection model to count the number of passengers present. The resulting count, accompanied by a detection confidence score, is submitted to `POST /api/v1/hardware/camera` via HTTP [7]. The backend's `ingest_camera()` handler persists a `CameraReading` record through SQLAlchemy [2] and computes a `crowding_score` as the ratio of `passenger_count` to the vehicle's rated `capacity` .

Two threshold levels govern the system's response:

- **`crowding_score ≥ 0.70`** : A `CrowdingEvent` record is written in PostgreSQL [10] for analytics and reporting. The associated trip's `crowding_score` and `is_crowded` flag are updated.
- **`crowding_score > 0.90`** : The automated extra-dispatch workflow is triggered. The system identifies the `DRIVER_3` position holder in the current active `RotationAssignment` for the affected route, creates a new `Trip` record for a relief bus with `is_extra_dispatch=True` , updates the vehicle status, and broadcasts a `crowding_alert` WebSocket [8] event via Redis [11] to Manager and Admin clients.

This closed-loop automation — from YOLOv8 [13] camera reading to database record to dispatched trip to operator notification — represents the system's most sophisticated hardware-software integration workflow and demonstrates the practical utility of the IoT architecture for operational bus management.

4.5.6 Hardware Authentication and Rate Limiting

All four hardware ingestion endpoints are protected by API key authentication implemented in the FastAPI [1] dependency layer. The hardware API key, configured through the `HARDWARE_API_KEY` environment variable, is a cryptographically generated secret distributed to all authorized hardware

nodes during provisioning. Each hardware request must include this key in the `X-Hardware-API-Key` HTTP [7] header; requests without a valid key receive HTTP 401 responses [7] and are logged.

Hardware endpoints are subject to dedicated rate limits configured through the **slowapi** rate-limiting middleware — for example, GPS endpoints permit up to 120 requests per minute per device, reflecting a maximum update frequency of 2 Hz — preventing a malfunctioning hardware node from flooding the PostgreSQL [10] database with sensor records.

4.6 Application Design

4.6.1 Backend Application Design

4.6.1.1 FastAPI Application Structure

The backend application is initialized in `/app/main.py` using FastAPI's [1] `lifespan` context manager pattern, which is the modern replacement for `startup / shutdown` event handlers. On application startup, the lifespan handler establishes the async database session factory via SQLAlchemy [2], initializes the Redis [11] connection pool, starts the APScheduler instance, and launches the Redis [11] Pub/Sub supervisor coroutine for WebSocket [8] broadcasting. This structured startup sequence ensures that all stateful dependencies are healthy before the server begins accepting traffic.

The application is organized into a layered module hierarchy:

```

app/
├─ main.py          (application factory, middleware registration)
├─ api/
│   ├─ auth.py      (authentication routes)
│   ├─ hardware.py   (IoT ingestion routes)
│   ├─ drivers.py    (driver operation routes)
│   ├─ admin.py      (administrator routes)
│   ├─ manager.py    (fleet manager routes)
│   ├─ websockets.py (WebSocket endpoint)
│   └─ deps.py       (shared dependency injections)
├─ models/
│   └─ models.py     (SQLAlchemy ORM model definitions)
├─ services/
│   ├─ rotation_service.py
│   ├─ break_service.py
│   ├─ trip_service.py
│   ├─ crowding_service.py
│   ├─ maintenance_service.py
│   ├─ notification_service.py
│   ├─ audit_service.py
│   └─ report_service.py
│   └─ scheduler.py
├─ middleware/
│   ├─ sanitization.py
│   └─ idempotency.py
└─ core/
    ├─ config.py     (Pydantic Settings)
    ├─ security.py    (bcrypt, JWT)
    ├─ sockets.py     (ConnectionManager)
    └─ rate_limit.py  (slowapi configuration)

```

Each route module is registered on the main FastAPI [1] application as an `APIRouter` with a common prefix (e.g., `prefix="/api/v1/admin"`) and a tag label for Swagger grouping. This prevents route prefix repetition in individual handler decorators and makes the overall route table inspectable at the router level.

4.6.1.2 Async Database Operations

All database interactions are performed through SQLAlchemy's [2] `AsyncSession`, obtained via a FastAPI [1] dependency (`get_db`). This pattern ensures that each request receives its own session scope, that sessions are committed on success and rolled back on exception, and that the underlying

`asyncpg` connection pool connected to PostgreSQL [10] is shared across all concurrent requests rather than establishing a new connection per request.

The canonical pattern employed throughout the codebase follows the **Unit of Work** model: a single `async with session.begin()` block wraps all related reads and writes within a given request handler, guaranteeing atomicity. For example, the ANPR ingestion handler creates a `GateLog` record, conditionally updates a `vehicle` status, and publishes a WebSocket [8] notification — all within a single SQLAlchemy [2] transaction, so partial state cannot persist if any step fails.

4.6.1.3 Middleware Stack

The application's middleware stack, registered in `main.py` in order of execution, consists of:

- **CORS Middleware:** Configured with origins read from `BACKEND_CORS_ORIGINS` environment variable, allowing the React [16] frontend's development server and production origin to communicate with the API via HTTP [7].
- **Sanitization Middleware** (`/middleware/sanitization.py`): Intercepts all incoming requests and passes string values through the **bleach** HTML-stripping library, neutralizing any **XSS** payloads embedded in request bodies before they reach route handlers.
- **Idempotency Middleware** (`/middleware/idempotency.py`): For state-mutating requests that carry an `Idempotency-Key` header, the middleware stores the response in Redis [11] with a short TTL. If a duplicate request with the same key arrives within the TTL window, the cached response is returned immediately without re-executing the handler, preventing double-submission of critical operations such as trip starts and maintenance requests.
- **Rate Limiting:** Applied per-endpoint via **slowapi** decorators on FastAPI [1] route handlers. Sensitive endpoints such as login are restricted to 5 requests per minute; hardware ingestion endpoints permit higher throughput commensurate with sensor polling frequencies.

4.6.1.4 Background Scheduler

The **APScheduler** `AsyncIOScheduler` instance, configured in `/app/services/scheduler.py`, manages three recurring background jobs:

1. **Daily Schedule Generation** (`generate_daily_schedule`): Fires at **05:30 AM** each day. Queries active routes via SQLAlchemy [2], identifies available `OFF_DUTY` drivers and `FREE` vehicles, and invokes `rotation_service.generate_daily_schedule()` to construct the full day's `RotationAssignment` and `Trip` records in PostgreSQL [10].
2. **Rotation Processing** (`rotation_manager_job`): Fires every **5 minutes**. Evaluates active rotation assignments against the current time, determines whether any driver exchange events are due,

and executes the exchange logic — updating driver statuses through SQLAlchemy [2], creating `DriverExchange` records, and triggering replacement trip assignments.

3. **Midnight Reset** (`midnight_reset_job`): Fires at **00:00**. Resets per-driver daily counters (`total_trips_today` , `break_time_remaining` , `trips_since_last_break`) and aggregates the day's operational metrics into a `DailyReport` record in PostgreSQL [10] for historical analytics.

4.6.2 Database Schema Design

4.6.2.1 Entity-Relationship Overview

The database schema, managed through Alembic migrations and defined in `/app/models/models.py` using SQLAlchemy [2], comprises **18 model classes** organized into five semantic groups within PostgreSQL [10]:

1. **Identity and Access**: `User` (authentication credentials, role assignment)
2. **Fleet Entities**: `Garage` , `Vehicle` , `Driver` , `Route` , `RouteStop`
3. **Operational Records**: `RotationAssignment` , `Trip` , `Ticket` , `DriverExchange` , `BreakLog`
4. **IoT Telemetry**: `GPSTracking` , `IotSensorReading` , `GateLog` , `CameraReading` , `CrowdingEvent` , `GateCamera`
5. **System Metadata**: `Notification` , `AuditLog` , `DailyReport` , `MaintenanceRequest` , `RerouteLog`

4.6.2.2 Key Relationships and Constraints

The `Driver` model maintains a one-to-one relationship with `User` (through a `user_id` foreign key enforced by PostgreSQL [10]) and carries a rich set of operational state columns designed to support the break management and rotation algorithms without requiring cross-table joins during time-critical scheduler runs. Columns such as `trips_since_last_break` , `break_time_remaining` , `fatigue_score` , `current_shift` , and `current_break_number` are maintained in place, trading normalization purity for query performance in the hot path of the rotation processor.

The `Trip` model links a `Driver` , `Vehicle` , `Route` , and `RotationAssignment` in a single SQLAlchemy [2] record and carries scheduling (`scheduled_start` , `scheduled_end`) and actuality (`actual_start` , `actual_end`) timestamps, enabling on-time performance calculation as `is_late = actual_end > scheduled_end + LATE_THRESHOLD_MINUTES` .

The `AuditLog` model uses PostgreSQL's [10] `JSONB` type for its `old_values` and `new_values` columns, allowing it to record arbitrary entity state snapshots — including nested relationships — without requiring a separate schema change each time a SQLAlchemy [2] model evolves. This design is documented in `/app/services/audit_service.py` , where `log_action()` serializes the before/after state of any affected entity into these `JSONB` columns alongside the acting user's ID and IP address.

4.6.2.3 Enumerated Types

SQLAlchemy [2] Enum types are used extensively to constrain the valid values of status fields at the PostgreSQL [10] database level, providing a second layer of integrity validation beyond Pydantic's [14] API-boundary checks. Key enumerations include:

- `UserRole` : ADMIN , MANAGER , DRIVER
- `DriverStatus` : ACTIVE , ON_TRIP , ON_BREAK , OFF_DUTY
- `VehicleStatus` : FREE , ASSIGNED , EN_ROUTE , MAINTENANCE , OUT_OF_SERVICE
- `TripStatus` : SCHEDULED , ACTIVE , COMPLETED , CANCELLED
- `TripDirection` : OUTBOUND , INBOUND
- `ShiftType` : MORNING , EVENING
- `IotSensorType` : ENGINE_TEMP , OIL_PRESSURE , BRAKE_PAD , BATTERY_VOLTAGE
- `MaintenanceStatus` : PENDING , APPROVED , IN_PROGRESS , COMPLETED , REJECTED
- `RerouteStatus` : PENDING , APPROVED , REJECTED

4.6.3 Frontend Application Design

4.6.3.1 Technology Stack and Justification

The client-side application is implemented as a **React 19** [16] single-page application, bundled by **Vite 7** [15]. React [16] is chosen for its component model, which maps naturally to the system's role-differentiated dashboard structure: each role (Admin, Manager, Driver) is represented as a subtree of components sharing a layout wrapper but branching into independent page modules. Vite [15] replaces Create React App as the build toolchain due to its native ES module development server, which provides sub-second hot module replacement, and its Rollup-based production build, which yields optimized chunk-split bundles.

Material-UI (MUI) 7 provides the design system: a comprehensive set of pre-built, accessible React [16] components (tables, dialogs, form inputs, snackbars) that conform to Material Design specifications. Using MUI eliminates the need to implement and test low-level UI primitives from scratch, allowing development effort to concentrate on business logic and data wiring.

4.6.3.2 Application Routing and Role-Based Access Control

Application routing is managed by **React Router DOM 7**. The route tree is structured to enforce authentication and RBAC at the layout level through the `ProtectedRoute` component (`/components/Layout/ProtectedRoute.jsx`). This component reads the authenticated user's role from the `AuthContext` and redirects unauthenticated users to `/login` and users attempting to access a

role-restricted route to the `/unauthorized` page, without requiring each individual page component to implement its own access check.

The route hierarchy follows the pattern:

```
/ (root redirect)
/login
/unauthorized
/admin/*    → ProtectedRoute[role=ADMIN]    → MainLayout → Admin pages
/manager/*  → ProtectedRoute[role=MANAGER]  → MainLayout → Manager pages
```

Driver-facing functionality is not present in the React web application; it is exclusively served by the Flutter mobile application (see Section 4.6.4).

The `Sidebar` component (`/components/Layout/Sidebar.jsx`) reads the current user's role from the JWT [9] payload stored in `AuthContext` and renders only the navigation entries appropriate to that role, ensuring that the navigation structure reflects access permissions without requiring users to discover restricted routes by trial and error.

4.6.3.3 State Management with Zustand

Global application state is managed through **Zustand 5**, a lightweight, hook-based state management library. Zustand is chosen over Redux for its significantly reduced boilerplate: stores are defined as plain JavaScript objects with setter functions, without requiring actions, reducers, or middleware configuration. Two Zustand stores are defined:

- **alertStore** (`/src/store/alertStore.js`): Maintains a circular array of up to 50 real-time alert objects received via WebSocket [8]. The `addAlert()` action appends new alerts (from `crowding_alert`, `iot_alert`, `gate_auth`, `speed_alert` WebSocket [8] message types) and evicts the oldest entry when the array is full. This store is consumed by the notification bell component in the `Header` and by the Manager's dedicated Notifications page.
- **settingsStore** (`/src/store/settingsStore.js`): Persists user interface preferences including text direction (**LTR/RTL** for Arabic/English bilingual support), active language, and theme configuration. These settings survive page refreshes through Zustand's `persist` middleware, which serializes the store to `localStorage`.

4.6.3.4 HTTP Communication Layer

All REST API communication is routed through a centralized **Axios** HTTP [7] instance defined in `/src/api/axios.js`. The instance is configured with the backend base URL and a request interceptor that reads the JWT [9] access token from `localStorage` and injects it as an

Authorization: Bearer <token> header on every outgoing HTTP [7] request. A response interceptor handles HTTP 401 responses [7] by clearing all authentication state from `localStorage` and redirecting the browser to `/login?session_expired=true`, providing a seamless re-authentication experience when JWT [9] tokens expire.

4.6.3.5 Real-Time WebSocket Integration

The `WebSocketContext` (`/src/context/WebSocketContext.jsx`) establishes and maintains the persistent WebSocket [8] connection to `ws(s)://[host]/ws`. The JWT [9] access token is transmitted in the WebSocket [8] handshake (either as a query parameter or in the `Sec-WebSocket-Protocol` header, per RFC 6455 [8]). On receiving a message, the context parses the JSON payload and dispatches it to the appropriate Zustand store action or React [16] state setter. Components that need to respond to real-time events — such as the Manager dashboard's fleet status panel or the crowding alert banner — subscribe to the relevant context values and re-render reactively when new data arrives from the Redis [11] Pub/Sub broadcast, without polling the REST API.

4.6.3.6 Internationalization and Session Management

The application supports bilingual operation in English and Arabic through a translation system implemented in `/src/i18n/translations.js` and accessed via the `useTranslation` hook. Arabic mode activates RTL text direction, managed through the `settingsStore`'s `direction` field and applied to the MUI theme's `direction` property, causing the entire React [16] component tree to mirror correctly.

An idle session timeout is enforced through the `useIdleTimer` custom hook (`/src/hooks/useIdleTimer.js`), which monitors user activity (mouse movement, keyboard input) and displays the `IdleWarningDialog` modal after a configured inactivity period, prompting the user to confirm their session before automatically logging them out and invalidating their JWT [9].

4.6.4 Mobile Application Design (Flutter — Bus Drivers)

4.6.4.1 Technology Stack and Justification

The driver-facing application is implemented in **Dart** using the **Flutter** SDK, targeting Android mobile devices. Flutter is chosen because it enables a single Dart codebase to produce a native-performance mobile application, eliminating the need to maintain separate Android and iOS codebases. Its widget-based rendering model, backed by the Skia/Impeller graphics engine, provides smooth UI animations independent of the platform's native widget toolkit.

BLoC (Business Logic Component), via the `flutter_bloc 8.1.6` package, is chosen as the state management architecture. BLoC enforces a strict separation between UI widgets (which emit events) and business logic classes (which consume events and emit states), making the application's data

flow explicit, testable, and consistent. The application defines nine Cubits: `AuthCubit` , `ForgotPasswordCubit` , `DashboardCubit` , `RoutesCubit` , `MessagesCubit` , `ProfileCubit` , `TripHistoryCubit` , `TripDetailsCubit` , and `MaintenanceCubit` .

4.6.4.2 HTTP Communication — DioHelper

All REST API communication is channelled through a static utility class, `DioHelper` (`lib/core/network/dio_helper.dart`), built on the **Dio 5.7.0** HTTP client. `DioHelper.init()` is called once at application start, configuring the base URL, a 15-second connect/receive timeout, and a JSON Content-Type default. Authenticated requests attach the JWT access token manually on each call:

```
// lib/core/network/dio_helper.dart
static Future<Response> getData({
  required String url,
  String? token,
  Map<String, dynamic>? query,
}) async {
  if (token != null)
    dio.options.headers['Authorization'] = 'Bearer $token';
  return await dio.get(url, queryParameters: query);
}
```

This per-request token injection differs from the React application's global Axios interceptor approach and reflects the current stage of the Flutter codebase; a unified Dio interceptor is the recommended refactoring target.

4.6.4.3 Token Persistence — CacheHelper

Authentication tokens are persisted across application restarts using **SharedPreferences 2.3.3**, wrapped in a `CacheHelper` class (`lib/core/utils/cache_helper.dart`). The JWT access token is stored under the key defined by `AppConstants.token` . On application startup, `AuthCubit` reads this key; if a valid token exists, the driver is considered authenticated and the navigation stack opens directly to the home dashboard.

4.6.4.4 Application Screens

The Flutter application provides the following screens, each driven by its corresponding BLoC Cubit:

Screen	Cubit	Key Functionality
Login	AuthCubit	POST /auth/login , token persistence

Screen	Cubit	Key Functionality
Forgot Password	ForgotPasswordCubit	Password reset flow
Home / Dashboard	DashboardCubit	Today's trip summary, status overview
Routes	RoutesCubit	Assigned route map and stop list
Messages	MessagesCubit	Manager–driver messaging
Profile	ProfileCubit	Driver profile view and edit
Trip History	TripHistoryCubit	Past trip records
Trip Details	TripDetailsCubit	Per-trip stats and timeline
Maintenance	MaintenanceCubit	Submit and track maintenance requests

4.6.4.5 Real-Time Notifications (Planned)

The current release does not implement a WebSocket connection in the Flutter app. Drivers receive updates by polling REST endpoints via BLoC state refresh triggers. A future implementation will introduce a `WebSocketService` using the `web_socket_channel` package, connecting to the backend's `/ws` endpoint with the driver's JWT token and routing incoming events into the relevant Cubits.

4.7 System Scenarios (Use Cases & Workflows)

4.7.1 Scenario 1: IoT-Triggered Emergency Maintenance Request

Context: A bus operating on an active trip has its engine temperature rise above the 105 °C safety threshold. The on-vehicle IoT sensor node built around the ESP32 [5] microcontroller detects this condition and submits a reading to the backend.

Step-by-step Workflow:

1. **Hardware Trigger:** The ESP32 [5] reads the engine thermocouple via its GPIO peripheral interface [5] and obtains a value of 107.4 °C. It packages this as a JSON payload:

```

{
  "vehicle_id": 12,
  "engine_temp_c": 107.4,
  "oil_pressure_psi": 42.1,
  "brake_pad_mm": 7.2,
  "battery_voltage": 12.6
}

```

The node issues `POST /api/v1/hardware/iot` over HTTP [7] with the `X-Hardware-API-Key` header.

2. **API Key Validation:** The `ingest_iot()` handler's FastAPI [1] dependency verifies the API key against the configured `HARDWARE_API_KEY` value. The request proceeds.
3. **Database Write:** An `IotSensorReading` record is inserted via SQLAlchemy [2] for `sensor_type=ENGINE_TEMP`, `value=107.4`, `vehicle_id=12`, `recorded_at=<now>` in PostgreSQL [10].
4. **Threshold Evaluation:** The handler compares each submitted value against the configured thresholds. `engine_temp_c=107.4 > 105` evaluates as `True`.
5. **Auto-Maintenance Creation:** The `maintenance_service` is invoked to create a `MaintenanceRequest` record with `type=EMERGENCY`, `status=PENDING`, `priority=1`, `title="Engine Temperature Critical"`, `vehicle_id=12` via SQLAlchemy [2]. The vehicle's status is updated to `OUT_OF_SERVICE` in PostgreSQL [10].
6. **Audit Logging:** The `audit_service.log_action()` records the vehicle status change and maintenance request creation in the `AuditLog` table with the old and new field values serialized as PostgreSQL [10] JSONB.
7. **WebSocket Broadcast:** The `ConnectionManager.broadcast_to_role()` method publishes an `iot_alert` message to the Redis [11] Pub/Sub channel. The Redis [11] subscriber delivers it to all `MANAGER` and `ADMIN` WebSocket [8] connections.
8. **Frontend Response:** On the Manager's browser, the React [16] `WebSocketContext` receives the `iot_alert` message, calls `alertStore.addAlert()`, and the notification bell count increments. The Manager's active fleet map tile for Vehicle 12 updates its status indicator to `OUT_OF_SERVICE` in real time.
9. **HTTP Response:** The `ingest_iot()` handler returns `HTTP 200` [7] with a JSON body confirming the reading was processed and indicating that an emergency maintenance request was created.

Total estimated latency from sensor reading to browser notification: < 2 seconds (dominated by HTTP [7] round-trip and Redis [11] Pub/Sub delivery).

4.7.2 Scenario 2: Driver Starts a Scheduled Trip

Context: A driver assigned to the morning shift's first route checks in at the garage, finds their assigned trip in the application, and presses "Start Trip" at T-5 minutes before the scheduled departure.

Step-by-step Workflow:

1. **Driver Login:** The driver submits `POST /api/v1/auth/login` with email and password via HTTP [7]. The `login()` handler verifies the bcrypt hash, generates a signed JWT [9] access token (60-minute expiry, HS256-signed [9]) and a refresh token (7-day expiry), and returns both.
2. **Check-In:** The driver calls `POST /api/v1/drivers/me/check-in`. The `check_in()` handler, protected by FastAPI's [1] JWT [9] dependency, verifies that the driver's `status` is `OFF_DUTY` in PostgreSQL [10] and that there is a valid `RotationAssignment` for today's date and the current shift. The driver's `status` is updated to `ACTIVE` via SQLAlchemy [2].
3. **Trip Retrieval:** The driver calls `GET /api/v1/drivers/me/trips` via HTTP [7], which returns their scheduled `Trip` records for the day. The response, validated by Pydantic [14] schemas, includes the trip's `id`, `scheduled_start`, `status=SCHEDULED`, route details, and assigned `vehicle`.
4. **Trip Start Request:** The driver submits `POST /api/v1/drivers/me/trips/{trip_id}/start` via HTTP [7]. The `start_trip()` handler executes the following validation sequence:
 - Confirms the driver's current status is `ACTIVE` (not `ON_BREAK` or `ON_TRIP`) via SQLAlchemy [2] query against PostgreSQL [10].
 - Confirms no other trip for this driver is currently in `ACTIVE` status.
 - Confirms `current_time ≥ scheduled_start - 15 minutes` (the 15-minute pre-start window). If the driver attempts to start more than 15 minutes early, HTTP 400 [7] is returned.
 - Confirms the driver has completed at least `MIN_TRIPS_BEFORE_BREAK` trips since their last break (this check is omitted for the first trip of the day).
5. **State Transitions:** On successful validation via SQLAlchemy [2]:
 - The `Trip` record's `status` is updated to `ACTIVE`, `actual_start` is set to the current timestamp in PostgreSQL [10].
 - The `Driver` record's `status` is updated to `ON_TRIP`, `current_vehicle_id` and `current_route_id` are set.
 - The `Vehicle` record's `status` is updated to `EN_ROUTE`.
6. **Audit and Notification:** `audit_service.log_action()` records the trip state transition in the PostgreSQL [10] `audit_logs` table. A `Notification` record is created for the manager role indicating that the trip has commenced.
7. **Response:** The FastAPI [1] handler returns `HTTP 200` [7] with the updated trip record, validated by Pydantic [14], including `actual_start`.

8. **Driver UI Update:** The Dio HTTP response is processed by the `TripDetailsCubit` inside the Flutter mobile application. The Cubit emits a new state carrying the updated `Trip` record, and the trip detail screen rebuilds — the status label updates from "Scheduled" to "In Progress" and the "Start Trip" button is replaced with an "End Trip" button.

4.7.3 Scenario 3: ANPR Gate Authorization for Returning Bus

Context: A bus completing its route returns to the garage. The ANPR camera at the exit gate captures the license plate as the bus approaches.

Step-by-step Workflow:

1. **Camera Capture:** The ANPR camera unit performs on-device optical character recognition on the captured frame, producing `plate_number="ABC-1234"` with `confidence=0.94`.
2. **API Submission:** The camera node issues `POST /api/v1/hardware/anpr` via HTTP [7] with the payload:

```
{
  "gate_id": "GATE_EXIT_01",
  "plate_number": "ABC-1234",
  "confidence": 0.94,
  "timestamp": "2025-05-01T14:32:10Z"
}
```

3. **Confidence Threshold Check:** The FastAPI [1] `ingest_anpr()` handler evaluates `confidence=0.94 ≥ 0.85`. The reading proceeds to authorization logic.
4. **Vehicle Lookup:** The handler queries the `vehicles` table via SQLAlchemy [2] against PostgreSQL [10] for `plate_number="ABC-1234"`. A matching `Vehicle` record is found with `status=EN_ROUTE`.
5. **Authorization Decision:** `EN_ROUTE` is in the permitted status set `{EN_ROUTE, ASSIGNED, FREE}`. The event is classified as `GRANTED`.
6. **Gate Log Write:** A `GateLog` record is inserted via SQLAlchemy [2] with `event=GRANTED`, `vehicle_id=<matched_id>`, `gate_id="GATE_EXIT_01"`, `plate_number="ABC-1234"`, `confidence=0.94` in PostgreSQL [10].
7. **State Update:** The handler updates the `Vehicle`'s `status` to `FREE` via SQLAlchemy [2], clearing `current_driver_id` and `current_route_id`.
8. **WebSocket Broadcast:** A `gate_auth` message is published to the Redis [11] Pub/Sub channel, containing the plate number, gate ID, authorization result, and vehicle details. All connected

Manager and Admin WebSocket [8] clients receive the event.

9. **HTTP Response:** The camera node receives `HTTP 200` [7] with the authorization result, which it can use to trigger the physical gate actuator (e.g., servo or relay-controlled boom barrier connected to the ESP32's [5] GPIO pins [5]).
10. **Manager Dashboard Update:** The React [16] Manager's fleet map updates Vehicle ABC-1234's status from `EN_ROUTE` to `FREE`, and the gate activity log panel appends the new `GRANTED` event entry in real time via the WebSocket [8] context.

4.8 Software Tools Used

The following tools, frameworks, and libraries constitute the complete technology stack of the Smart Bus Garage Management System. Each selection is justified in terms of its fitness for an enterprise-grade IoT graduation project.

4.8.1 Backend Technologies

Python 3.11+

Python is selected as the primary backend language due to its mature async ecosystem (`asyncio` , `asyncpg`), the breadth of its IoT and data-processing libraries, and its dominance in modern API development. Its readable syntax reduces onboarding time for future maintainers and aligns with the team's existing expertise.

FastAPI 0.115+ [1]

FastAPI [1] provides a high-performance, standards-based ASGI web framework with first-class support for async database operations, automatic OpenAPI/Swagger documentation, and Pydantic [14]-powered request validation. It is chosen over Flask or Django REST Framework due to its native async model, which is essential for concurrently handling high-frequency IoT sensor submissions without thread-blocking overhead.

SQLAlchemy 2.0 (Async) [2]

SQLAlchemy's [2] second-generation async API, introduced in version 2.0, provides a full-featured ORM with `asyncio` integration through the `asyncpg` driver for PostgreSQL [10]. It is chosen over raw SQL or a lighter ORM because its declarative model definition enables Alembic-managed schema migrations, relationship loading strategies, and typed column definitions — all critical for a system with 18 interrelated models.

asyncpg 0.30+

asyncpg is the highest-performance asynchronous PostgreSQL [10] driver available for Python, achieving throughput significantly exceeding that of psycopg2 in connection-pool scenarios. It is selected as the database connectivity layer to maximize the throughput of the hardware ingestion endpoints, which may receive hundreds of GPS readings per minute.

PostgreSQL 16 [10]

PostgreSQL [10] serves as the primary relational database. Its support for JSONB columns, comprehensive indexing options (including composite indexes on (vehicle_id, recorded_at) for time-series queries), and strong transactional guarantees make it the appropriate choice for a system that must maintain referential integrity across a complex entity graph while also storing semi-structured audit data.

Alembic 1.12+

Alembic provides version-controlled, incremental database schema migrations integrated with SQLAlchemy's [2] model definitions. It ensures that schema changes can be applied and reversed reproducibly across development, testing, and production environments.

Redis 5.0+ [11]

Redis [11] is employed as both a Pub/Sub message broker (for WebSocket [8] alert distribution across ASGI workers) and a JWT [9] token blacklist store (for post-password-change token invalidation). Its in-memory architecture provides sub-millisecond latency for both use cases.

APScheduler 3.10+

APScheduler's AsyncIOScheduler integrates directly with the asyncio event loop, allowing background jobs (daily schedule generation, rotation processing, midnight reset) to execute scheduled async coroutines without spawning separate threads or processes. This simplifies deployment by eliminating the need for an external job scheduler such as Celery Beat.

Pydantic 2.10+ [14]

Pydantic's [14] v2 Rust-accelerated validation engine enforces strict type checking and constraint validation on all API request and response schemas. Its deep integration with FastAPI [1] makes it the de facto standard for request model definition, providing automatic error messages for invalid payloads without requiring manual validation code.

python-jose / bcrypt

python-jose provides HS256 JWT [9] creation and verification; bcrypt provides password hashing with configurable work factors. Together they implement the authentication security layer in compliance with RFC 7519 [9] and industry-standard practices for credential storage and stateless session management.

slowapi 0.1.9+

slowapi ports the Flask-Limiter rate-limiting API to FastAPI [1]/Starlette, providing per-endpoint, per-IP, and per-user rate limits. It is used to protect authentication endpoints from brute-force attacks and hardware endpoints from accidental flooding.

bleach 6.1+

bleach is a Python HTML sanitization library used in the `SanitizationMiddleware` to strip or escape HTML tags and attributes from incoming request bodies, providing defense against stored **XSS** attacks as categorized by **OWASP**.

4.8.2 Frontend Technologies

React 19 [16]

React's [16] component model, unidirectional data flow, and ecosystem of supporting libraries make it the most widely adopted choice for building complex, role-differentiated single-page applications. Version 19's concurrent rendering capabilities improve perceived performance for data-heavy dashboard views.

Vite 7 [15]

Vite's [15] native ES module development server eliminates the bundling step during development, achieving near-instantaneous hot module replacement. Its Rollup production build produces aggressively tree-shaken, chunk-split output, reducing the initial load time of the application's JavaScript payload.

React Router DOM 7

React Router provides declarative client-side routing with a nested route model, enabling the system's role-based layout hierarchy (public routes, protected admin/manager/driver subtrees) to be expressed as a composable React [16] component tree rather than imperative navigation logic.

Zustand 5

Zustand's minimal API (stores as plain JavaScript objects with hook-based access, no reducers or action creators required) reduces global state management boilerplate to near zero compared to Redux. Its `persist` middleware enables transparent `localStorage` synchronization for user preference settings without custom serialization code.

Material-UI (MUI) 7

MUI provides a comprehensive, accessible, and highly customizable React [16] component library implementing the Material Design specification. Its theming system allows consistent visual branding

across all role-specific dashboards, and its support for RTL text direction (Arabic) through the `direction` property of the MUI theme is essential for the system's bilingual requirement.

Axios 1.13+

Axios is the de facto standard HTTP [7] client for React [16] applications, offering a clean interceptor API for JWT [9] injection, response normalization, and centralized error handling. Its interceptor model allows the application's token refresh and session-expiry logic to be implemented once in `/src/api/axios.js` rather than repeated in every API call site.

React Leaflet / Leaflet 1.9

React Leaflet wraps the Leaflet.js interactive mapping library as React [16] components, enabling GPS-tracked vehicle positions to be rendered on live maps within the Manager's fleet dashboard. Leaflet is chosen for its permissive licensing, strong OpenStreetMap integration, and extensive plugin ecosystem.

Recharts 3.7

Recharts provides a declarative, SVG-based charting library for React [16]. It is used in the reporting and analytics screens to render trip completion rates, crowding score trends, and revenue figures as responsive, interactive charts.

Framer Motion 12

Framer Motion provides a declarative animation API for React [16]. It is used in the `PageTransition` component to apply smooth enter/exit animations between routes, and in modal transitions, improving the perceived fluidity of the user interface.

Dart 3 / Flutter SDK 3.x

Dart is a strongly-typed, garbage-collected language with first-class `async/await` support and a compile-to-native toolchain. Flutter compiles Dart to ARM native code for production Android builds, achieving performance comparable to hand-written Kotlin while sharing a single codebase. Flutter's widget model — where every UI element is an immutable, composable widget — maps naturally to the BLoC pattern's state-driven rebuild model.

flutter_bloc 8.1.6

`flutter_bloc` provides the `BlocProvider`, `BlocBuilder`, and `BlocListener` widgets that wire Cubit state streams to the Flutter widget tree. It is chosen over `Provider` or `Riverpod` for its strict event-driven architecture, which enforces testability and explicit data flow tracing.

Dio 5.7.0

Dio is a feature-rich Dart HTTP client with support for request cancellation, timeouts, interceptors, and

multipart form uploads. It is used in `DioHelper` as the sole HTTP transport layer for all driver-facing API communication.

SharedPreferences 2.3.3

SharedPreferences provides a key-value persistent storage layer backed by the platform's native preferences mechanism (Android `SharedPreferences`). It is used in `CacheHelper` to persist the JWT access token and user ID between application sessions.

4.8.3 Hardware and IoT Technologies

ESP32 Microcontroller (Espressif Systems) [5]

The ESP32 [5] dual-core Xtensa LX6 microcontroller is the hardware platform for vehicle-mounted sensor nodes, chosen for its integrated dual-band Wi-Fi and Bluetooth radio, rich GPIO set [5], support for I2C/SPI/UART peripheral interfaces, and widespread availability of Arduino-framework libraries for GPS, CAN bus, and analog sensing applications. Its 240 MHz clock speed and 520 KB SRAM are sufficient to handle sensor polling, JSON payload construction, and HTTP [7] POST transmission concurrently.

Arduino Framework / ESP-IDF [6]

The Arduino framework for ESP32 [5] abstracts the hardware peripheral register configuration, enabling rapid development of sensor reading and HTTP [7] communication firmware using the well-documented Arduino API. For production firmware, Espressif's native ESP-IDF [6] SDK can be used for finer control over power management and RTOS task scheduling.

YOLOv8 (Ultralytics) [13]

YOLOv8 [13] is used for real-time object detection in the passenger-counting subsystem. Its nano and small model variants can execute inference on edge compute hardware (e.g., NVIDIA Jetson Nano or a co-processor alongside the ESP32 [5] gateway) at 30+ FPS, making it suitable for real-time occupancy estimation in a moving bus cabin.

MQTT Protocol [12]

While the current implementation uses HTTP [7] for hardware-to-backend communication for its simplicity and unified authentication model, MQTT [12] is recognized as an alternative IoT messaging protocol suitable for constrained devices with low-bandwidth connections. MQTT's [12] publish/subscribe model, Quality of Service levels, and persistent sessions make it a strong candidate for future hardware integration work where network reliability cannot be guaranteed.

4.8.4 Development and Deployment Infrastructure

Alembic CLI

Used for generating and applying incremental database migration scripts, ensuring that the PostgreSQL [10] schema stays synchronized with SQLAlchemy [2] model definitions across all environments.

Uvicorn

Uvicorn is the ASGI server used to run the FastAPI [1] application. Its `uvloop` event loop backend provides C-level performance for asyncio operations, and its support for multiple worker processes (via Gunicorn) enables horizontal scaling of the backend on multi-core hardware.

Git / GitHub

Version control is managed with Git, with the repository hosted on GitHub. Branch-based development and pull request reviews enforce code quality standards and provide a traceable history of all system changes.

List of Abbreviations

Abbreviation	Full Term
ANPR	Automatic Number Plate Recognition
API	Application Programming Interface
APScheduler	Advanced Python Scheduler
ASGI	Asynchronous Server Gateway Interface
bcrypt	Blowfish Crypt (password hashing function)
CAN	Controller Area Network
CLI	Command Line Interface
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets

Abbreviation	Full Term
DB	Database
DBMS	Database Management System
ESP32	Espressif Systems Platform 32-bit
ESP-IDF	Espressif IoT Development Framework
FPS	Frames Per Second
GPIO	General Purpose Input/Output
GPS	Global Positioning System
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
HS256	HMAC with SHA-256 (JWT signing algorithm)
HMAC	Hash-based Message Authentication Code
I2C	Inter-Integrated Circuit
IEEE	Institute of Electrical and Electronics Engineers
IoT	Internet of Things
IP	Internet Protocol
JSON	JavaScript Object Notation
JSONB	JavaScript Object Notation Binary (PostgreSQL type)
JWT	JSON Web Token
JTI	JWT ID (unique identifier claim within a JWT)
LTR	Left-To-Right
MQTT	Message Queuing Telemetry Transport
MUI	Material User Interface
MVC	Model-View-Controller

Abbreviation	Full Term
NTC	Negative Temperature Coefficient
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
PSI	Pounds per Square Inch
REST	Representational State Transfer
RFC	Request for Comments
RBAC	Role-Based Access Control
RTL	Right-To-Left
RTOS	Real-Time Operating System
SDK	Software Development Kit
SHA	Secure Hash Algorithm
SIM	Subscriber Identity Module
SPA	Single-Page Application
SPI	Serial Peripheral Interface
SQL	Structured Query Language
SRAM	Static Random-Access Memory
SVG	Scalable Vector Graphics
TCP	Transmission Control Protocol
TTL	Time To Live
UART	Universal Asynchronous Receiver-Transmitter
UI	User Interface
URL	Uniform Resource Locator
URI	Uniform Resource Identifier
UUID	Universally Unique Identifier

Abbreviation	Full Term
WS	WebSocket
WSS	WebSocket Secure
XSS	Cross-Site Scripting
YOLO	You Only Look Once

References

- [1] S. Ramírez, *FastAPI Documentation*, Tiangolo, 2024. [Online]. Available: <https://fastapi.tiangolo.com>. [Accessed: Apr. 20, 2026].
- [2] M. Bayer, *SQLAlchemy 2.0 Documentation*, SQLAlchemy Project, 2024. [Online]. Available: <https://docs.sqlalchemy.org/en/20/>. [Accessed: Apr. 20, 2026].
- [3] Google LLC, *Flutter Documentation: The UI Toolkit for Any Screen*, Google LLC, 2024. [Online]. Available: <https://flutter.dev/docs>. [Accessed: Apr. 20, 2026].
- [4] Google LLC, *Dart Programming Language Documentation*, Google LLC, 2024. [Online]. Available: <https://dart.dev/guides>. [Accessed: Apr. 20, 2026].
- [5] Espressif Systems, *ESP32 Technical Reference Manual*, Version 5.3, Espressif Systems Ltd., Shanghai, China, 2024. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf. [Accessed: Apr. 20, 2026].
- [6] Espressif Systems, *ESP-IDF Programming Guide*, Version 5.3, Espressif Systems Ltd., 2024. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/index.html>. [Accessed: Apr. 20, 2026].
- [7] R. Fielding, M. Nottingham, and J. Reschke, "HTTP Semantics," Internet Engineering Task Force (IETF), RFC 9110, June 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9110>. [Accessed: Apr. 20, 2026].
- [8] I. Fette and A. Melnikov, "The WebSocket Protocol," Internet Engineering Task Force (IETF), RFC 6455, Dec. 2011. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6455>. [Accessed: Apr. 20, 2026].

[9] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Engineering Task Force (IETF), RFC 7519, May 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>. [Accessed: Apr. 20, 2026].

[10] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, The PostgreSQL Global Development Group, 2024. [Online]. Available: <https://www.postgresql.org/docs/16/>. [Accessed: Apr. 20, 2026].

[11] Redis Ltd., *Redis Documentation*, Redis Ltd., 2024. [Online]. Available: <https://redis.io/docs/latest/>. [Accessed: Apr. 20, 2026].

[12] OASIS Standards, *MQTT Version 5.0*, OASIS Standard, OASIS Open, Burlington, MA, USA, Mar. 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>. [Accessed: Apr. 20, 2026].

[13] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics YOLOv8 Documentation*, Ultralytics Inc., 2023. [Online]. Available: <https://docs.ultralytics.com>. [Accessed: Apr. 20, 2026].

[14] S. Colvin *et al.*, *Pydantic V2 Documentation*, Pydantic, 2024. [Online]. Available: <https://docs.pydantic.dev/latest/>. [Accessed: Apr. 20, 2026].

[15] E. You *et al.*, *Vite: Next Generation Frontend Tooling — Official Documentation*, Vite Project, 2024. [Online]. Available: <https://vitejs.dev/guide/>. [Accessed: Apr. 20, 2026].

[16] Meta Platforms, Inc., *React: The Library for Web and Native User Interfaces*, Meta Platforms, Inc., 2024. [Online]. Available: <https://react.dev>. [Accessed: Apr. 20, 2026].